

Master's thesis

Amortized Multi-Verifier Proofs

Implementation and Benchmarks

Antonio Peso Vilella

Tutors: Nikitas Paslis and Javier Herranz

Master in Advanced Mathematics and Mathematical Engineering (MAMME)

Facultat de Matemàtiques i Estadística · Universitat Politècnica de Catalunya



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH
Facultat de Matemàtiques i Estadística



Outline

1. Introduction
2. Thesis motivation
3. Relations, reductions and folding
4. The protocol
5. Experimental results
6. Conclusions

Proving that a computation is correct

A **Prover** $\mathcal{P}$ claims that a computation C gives

$$C(x, w) = y,$$

with x a public input and w a private input (the *witness*).

Proving that a computation is correct

A **Prover** $\mathcal{P}$ claims that a computation C gives

$$C(x, w) = y,$$

with x a public input and w a private input (the *witness*). A **Verifier** $\mathcal{V}$ wants to be convinced

- with **much less work** than running C itself,
- **without trusting** $\mathcal{P}$.

Proving that a computation is correct

A **Prover** $\mathcal{P}$ claims that a computation C gives

$$C(x, w) = y,$$

with x a public input and w a private input (the *witness*). A **Verifier** $\mathcal{V}$ wants to be convinced

- with **much less work** than running C itself,
- **without trusting** $\mathcal{P}$.

Natural when $\mathcal{P}$ is powerful (a server) and $\mathcal{V}$ is weak (a phone, a blockchain node).

Proving that a computation is correct

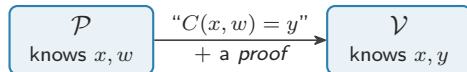
A **Prover** $\mathcal{P}$ claims that a computation C gives

$$C(x, w) = y,$$

with x a public input and w a private input (the *witness*). A **Verifier** $\mathcal{V}$ wants to be convinced

- with **much less work** than running C itself,
- **without trusting** $\mathcal{P}$.

Natural when $\mathcal{P}$ is powerful (a server) and $\mathcal{V}$ is weak (a phone, a blockchain node).



$\mathcal{V}$ never sees w and
does not re-run C

Interactive proofs

$\mathcal{P}$ and $\mathcal{V}$ exchange messages in **rounds**: $\mathcal{V}$ sends **random challenges**, $\mathcal{P}$ answers, and at the end $\mathcal{V}$ accepts or rejects.

Interactive proofs

$\mathcal{P}$ and $\mathcal{V}$ exchange messages in **rounds**: $\mathcal{V}$ sends **random challenges**, $\mathcal{P}$ answers, and at the end $\mathcal{V}$ accepts or rejects.

- **Completeness:** if the claim is true, an honest $\mathcal{P}$ always convinces $\mathcal{V}$.

Interactive proofs

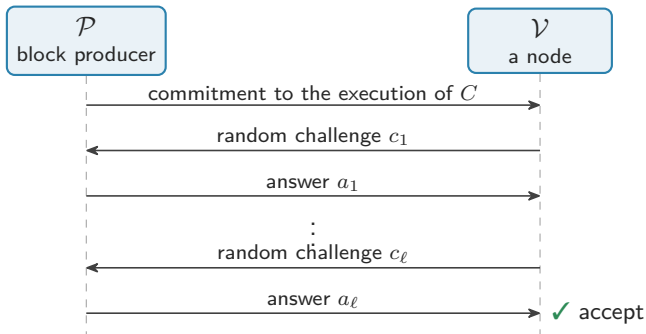
$\mathcal{P}$ and $\mathcal{V}$ exchange messages in **rounds**: $\mathcal{V}$ sends **random challenges**, $\mathcal{P}$ answers, and at the end $\mathcal{V}$ accepts or rejects.

- **Completeness:** if the claim is true, an honest $\mathcal{P}$ always convinces $\mathcal{V}$.
- **Soundness:** if the claim is false, *any* cheating $\mathcal{P}$ convinces $\mathcal{V}$ only with negligible probability.

Goldwasser, Micali, Rackoff (1985/89)

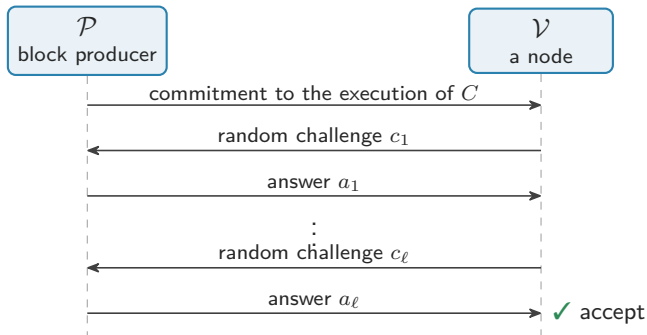
Example 1/3: an interactive proof of a transaction

tx: Alice $\rightarrow$ Bob, 5 coins **claim:** Alice signed it, has ≥ 5 coins,
and the new ledger is $S' = \text{Apply}(S, \text{tx})$



Example 1/3: an interactive proof of a transaction

tx: Alice $\rightarrow$ Bob, 5 coins **claim:** Alice signed it, has ≥ 5 coins, and the new ledger is $S' = \text{Apply}(S, \text{tx})$



✓ $\mathcal{V}$ does not re-execute the transaction.

But:

- ✗ $\mathcal{P}$ and $\mathcal{V}$ must be **online together** for many rounds.
- ✗ **One conversation per node:** 1000 nodes, 1000 proofs.
- ✗ The transcript convinces **nobody else**.

Succinct **N**on-interactive **AR**gument of **K**nowledge

Succinct Non-interactive **AR**gument of **K**nowledge

- **Succinct:** the proof π is tiny and $\mathcal{V}$ checks it in milliseconds, whatever the size of C .

Succinct Non-interactive **AR**gument of **K**nowledge

- **Succinct:** the proof π is tiny and $\mathcal{V}$ checks it in milliseconds, whatever the size of C .
- **Non-interactive:** a single message π .

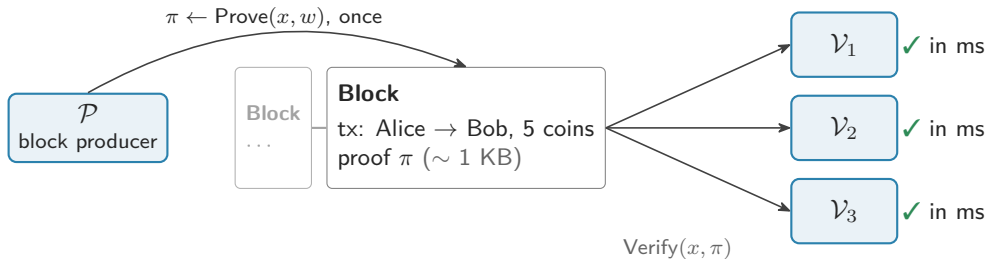
Succinct Non-interactive **AR**gument of **K**nowledge

- **Succinct:** the proof π is tiny and $\mathcal{V}$ checks it in milliseconds, whatever the size of C .
- **Non-interactive:** a single message π .
- **Argument:** sound against efficient provers (cryptographic assumptions).

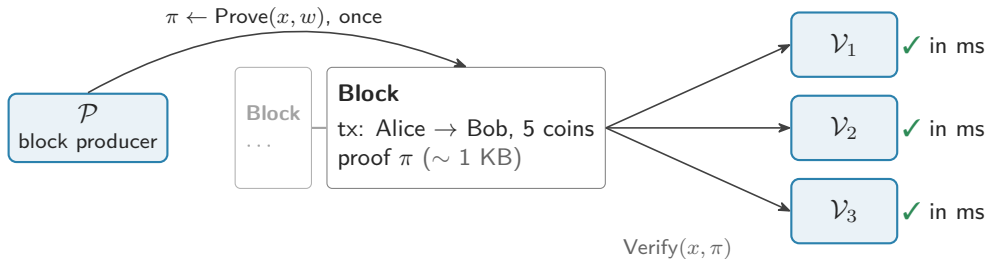
Succinct Non-interactive **AR**gument of **K**nowledge

- **Succinct:** the proof π is tiny and $\mathcal{V}$ checks it in milliseconds, whatever the size of C .
- **Non-interactive:** a single message π .
- **Argument:** sound against efficient provers (cryptographic assumptions).
- **of Knowledge:** a convincing $\mathcal{P}$ must actually *know* a witness w .

Example 2/3: the SNARK version



Example 2/3: the SNARK version



- ✓ No interaction: π travels with the transaction.
- ✓ Every node checks the **same** π , in milliseconds.

✗ Everything is **public**: who paid whom, and how much.

Zero-knowledge

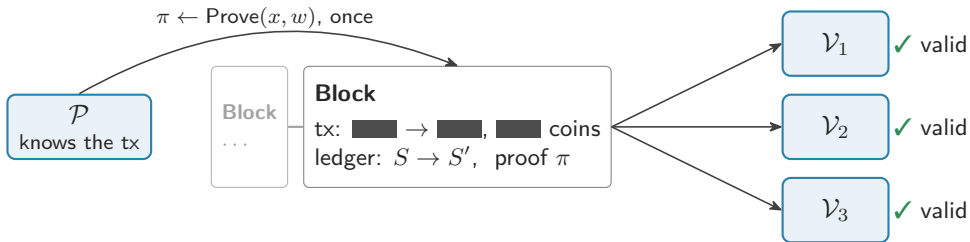
The proof reveals **nothing beyond the validity** of the claim.

Zero-knowledge

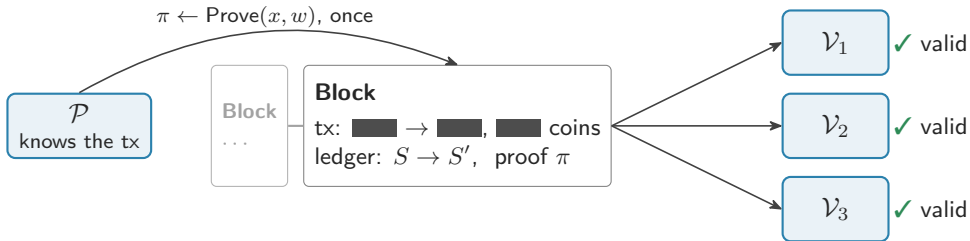
The proof reveals **nothing beyond the validity** of the claim.

zkSNARK = SNARK + zero-knowledge.

Example 3/3: the zero-knowledge SNARK version



Example 3/3: the zero-knowledge SNARK version



- ✓ Everything from the SNARK version.
- ✓ The chain only learns: *"a valid transaction took the ledger from S to S' "*.

Sender, receiver and amount stay **private**: the witness w .

Used for private payments (e.g. Zcash) and private smart contracts.

Outline

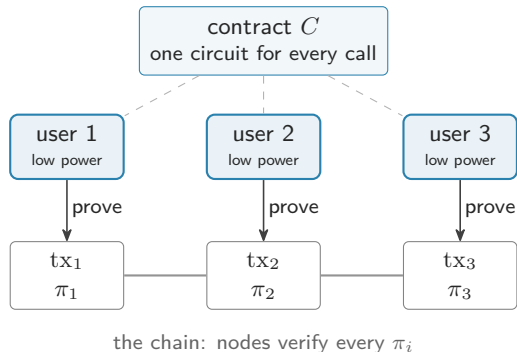
1. Introduction
2. Thesis motivation
3. Relations, reductions and folding
4. The protocol
5. Experimental results
6. Conclusions

A blockchain where every contract is proved by a SNARK

- A **smart contract** is a fixed program C : every call runs C on new inputs.
- Each call comes with a **SNARK** π : nodes check π instead of re-executing C .
- The same contract is called by **many independent users**, and each of them must produce a proof for C .
- Users run on **low-power devices** (phones, browsers): proving is the bottleneck.

A blockchain where every contract is proved by a SNARK

- A **smart contract** is a fixed program C : every call runs C on new inputs.
- Each call comes with a **SNARK** π : nodes check π instead of re-executing C .
- The same contract is called by **many independent users**, and each of them must produce a proof for C .
- Users run on **low-power devices** (phones, browsers): proving is the bottleneck.



Splitting the prover's work

Many SNARK provers (e.g. Marlin) split in two parts:

witness-dependent

uses the secret w :
stays with the user

witness-independent

public data only: **delegate** it

Splitting the prover's work

Many SNARK provers (e.g. Marlin) split in two parts:

witness-dependent

uses the secret w :
stays with the user

witness-independent

public data only: **delegate** it

- The witness-independent part only depends on the **contract** and on **public random challenges** α, β : it proves a claim $P(\alpha, \beta) = y$ about a polynomial P that encodes the contract.

Splitting the prover's work

Many SNARK provers (e.g. Marlin) split in two parts:

witness-dependent

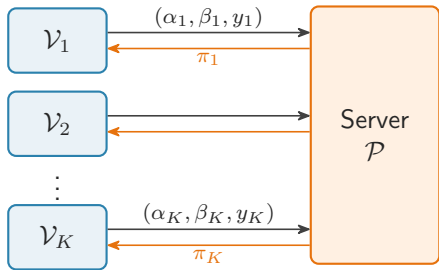
uses the secret w :
stays with the user

witness-independent

public data only: **delegate** it

- The witness-independent part only depends on the **contract** and on **public random challenges** α, β : it proves a claim $P(\alpha, \beta) = y$ about a polynomial P that encodes the contract.
- Delegating it to a **Server** reveals nothing private. Roles flip: the **user is the Verifier**, the **Server is the Prover**.

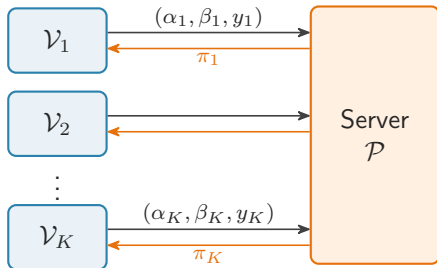
The Server's problem: amortization



The Server's problem: amortization

K users of the **same contract** send K statements on the **same** polynomial P :

$$P(\alpha_i, \beta_i) = y_i, \quad i = 1, \dots, K.$$

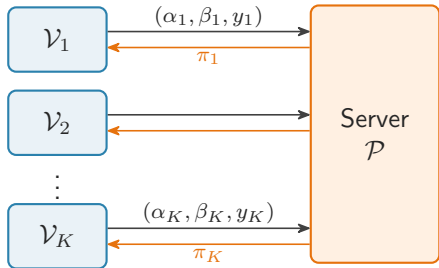


The Server's problem: amortization

K users of the **same contract** send K statements on the **same** polynomial P :

$$P(\alpha_i, \beta_i) = y_i, \quad i = 1, \dots, K.$$

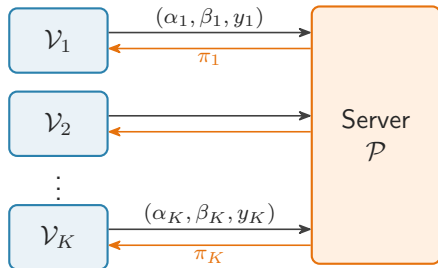
- **Naive:** K independent proofs, so the Server pays $K \times$ the cost of one proof.



The Server's problem: amortization

K users of the **same contract** send K statements on the **same** polynomial P :

$$P(\alpha_i, \beta_i) = y_i, \quad i = 1, \dots, K.$$



- **Naive:** K independent proofs, so the Server pays $K \times$ the cost of one proof.
- **Goal:** the Server pays about **one** proof plus a small overhead, and each user still verifies its own short proof, paying only a little more ($\approx \log K$).

Tool: *folding*, as in the protocol of Paslis, Ràfols and Zacharakis (ASIACRYPT 2026).

This thesis

1. **Analysis** of the amortization protocol of Paslis, Ràfols and Zacharakis: its building blocks, step by step, and its costs.

This thesis

1. **Analysis** of the amortization protocol of Paslis, Ràfols and Zacharakis: its building blocks, step by step, and its costs.
2. **Implementation** of the whole protocol in Rust, instantiated with the **Marlin** SNARK. (arkworks, BLS12-381, KZG commitments)

This thesis

1. **Analysis** of the amortization protocol of Paslis, Ràfols and Zacharakis: its building blocks, step by step, and its costs.
2. **Implementation** of the whole protocol in Rust, instantiated with the **Marlin** SNARK. (arkworks, BLS12-381, KZG commitments)
3. **Benchmarks**: Server and verifier costs against running K independent Marlin proofs.

This thesis

1. **Analysis** of the amortization protocol of Paslis, Ràfols and Zacharakis: its building blocks, step by step, and its costs.
2. **Implementation** of the whole protocol in Rust, instantiated with the **Marlin** SNARK. (arkworks, BLS12-381, KZG commitments)
3. **Benchmarks**: Server and verifier costs against running K independent Marlin proofs.
4. **Experiments that justify delegation**: how much of Marlin is witness-independent?

This thesis

1. **Analysis** of the amortization protocol of Paslis, Ràfols and Zacharakis: its building blocks, step by step, and its costs.
2. **Implementation** of the whole protocol in Rust, instantiated with the **Marlin** SNARK. (arkworks, BLS12-381, KZG commitments)
3. **Benchmarks**: Server and verifier costs against running K independent Marlin proofs.
4. **Experiments that justify delegation**: how much of Marlin is witness-independent?

Code: github.com/apesvil/amortized-proofs-masters-thesis

Outline

1. Introduction
2. Thesis motivation
3. Relations, reductions and folding
4. The protocol
5. Experimental results
6. Conclusions

Relations

Definition (relation)

A **relation** $\mathcal{R}$ is a set of pairs (x, w) : the **statement** x is public, the **witness** w is private. Its **language** is

$$\mathcal{L}(\mathcal{R}) = \{x : \exists w \text{ with } (x, w) \in \mathcal{R}\}.$$

Relations

Definition (relation)

A **relation** $\mathcal{R}$ is a set of pairs (x, w) : the **statement** x is public, the **witness** w is private. Its **language** is

$$\mathcal{L}(\mathcal{R}) = \{x : \exists w \text{ with } (x, w) \in \mathcal{R}\}.$$

A proof system for $\mathcal{R}$ convinces $\mathcal{V}$ that $x \in \mathcal{L}(\mathcal{R})$ (*of knowledge*: that $\mathcal{P}$ knows a w).

Relations

Definition (relation)

A **relation** $\mathcal{R}$ is a set of pairs (x, w) : the **statement** x is public, the **witness** w is private. Its **language** is

$$\mathcal{L}(\mathcal{R}) = \{x : \exists w \text{ with } (x, w) \in \mathcal{R}\}.$$

A proof system for $\mathcal{R}$ convinces $\mathcal{V}$ that $x \in \mathcal{L}(\mathcal{R})$ (*of knowledge*: that $\mathcal{P}$ knows a w).

Example: rank-1 constraint systems (R1CS)

Any computation can be written with matrices $\mathbf{A}, \mathbf{B}, \mathbf{C} \in \mathbb{F}^{n \times n}$ so that

$$(x, w) \in \mathcal{R}_{\text{R1CS}} \iff \mathbf{A}\mathbf{z} \circ \mathbf{B}\mathbf{z} = \mathbf{C}\mathbf{z}, \quad \mathbf{z} = (x, w),$$

with $\circ$ the entry-wise product. The matrices are fixed by the contract and **sparse**: $s_{\mathbf{M}} = \mathcal{O}(n)$ non-zero entries.

Tool: polynomial commitments (KZG)

$\text{Com}(p)$: a short **fingerprint** of a polynomial $p \in \mathbb{F}[X]$ of degree $< D$, a single elliptic-curve point (48 bytes).

Tool: polynomial commitments (KZG)

$\text{Com}(p)$: a short **fingerprint** of a polynomial $p \in \mathbb{F}[X]$ of degree $< D$, a single elliptic-curve point (48 bytes).

- **Binding:** it cannot later be opened to another polynomial.

Tool: polynomial commitments (KZG)

$\text{Com}(p)$: a short **fingerprint** of a polynomial $p \in \mathbb{F}[X]$ of degree $< D$, a single elliptic-curve point (48 bytes).

- **Binding:** it cannot later be opened to another polynomial.
- **Opening:** prove $p(x) = y$ with one more point.

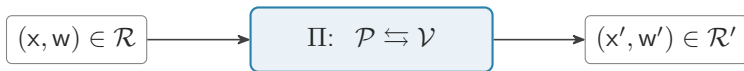
Tool: polynomial commitments (KZG)

$\text{Com}(p)$: a short **fingerprint** of a polynomial $p \in \mathbb{F}[X]$ of degree $< D$, a single elliptic-curve point (48 bytes).

- **Binding:** it cannot later be opened to another polynomial.
- **Opening:** prove $p(x) = y$ with one more point.
- **Additively homomorphic:** $\text{Com}(p) + r \cdot \text{Com}(q) = \text{Com}(p + r q)$, so many claims can be **merged** with a random r .

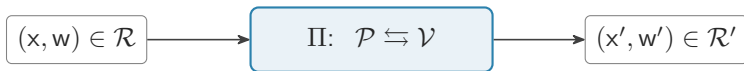
Kate, Zaverucha, Goldberg (ASIACRYPT 2010)

Reductions of knowledge



A **reduction** $\Pi: \mathcal{R} \rightarrow \mathcal{R}'$ is an interaction after which $\mathcal{V}$ outputs a new statement x' and $\mathcal{P}$ a new witness w' .

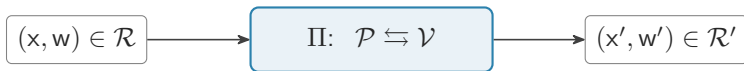
Reductions of knowledge



A **reduction** $\Pi: \mathcal{R} \rightarrow \mathcal{R}'$ is an interaction after which $\mathcal{V}$ outputs a new statement x' and $\mathcal{P}$ a new witness w' .

- **Completeness:** $(x, w) \in \mathcal{R} \implies (x', w') \in \mathcal{R}'$.

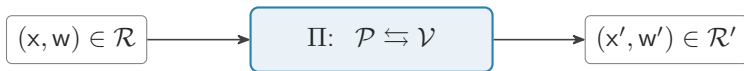
Reductions of knowledge



A **reduction** $\Pi : \mathcal{R} \rightarrow \mathcal{R}'$ is an interaction after which $\mathcal{V}$ outputs a new statement x' and $\mathcal{P}$ a new witness w' .

- **Completeness:** $(x, w) \in \mathcal{R} \implies (x', w') \in \mathcal{R}'$.
- **Knowledge soundness:** from anyone who knows a witness for x' , a witness for x can be extracted.
 $\implies$ **proving x' is enough to prove x .**

Reductions of knowledge



A **reduction** $\Pi : \mathcal{R} \rightarrow \mathcal{R}'$ is an interaction after which $\mathcal{V}$ outputs a new statement x' and $\mathcal{P}$ a new witness w' .

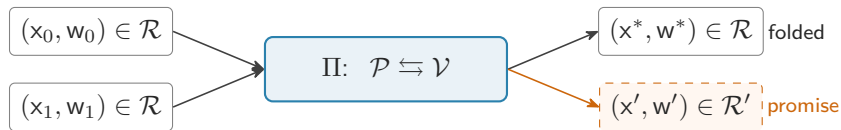
- **Completeness:** $(x, w) \in \mathcal{R} \implies (x', w') \in \mathcal{R}'$.
- **Knowledge soundness:** from anyone who knows a witness for x' , a witness for x can be extracted.
 $\implies$ **proving x' is enough to prove x .**

They compose like functions

Kothapalli, Parno (CRYPTO 2023)

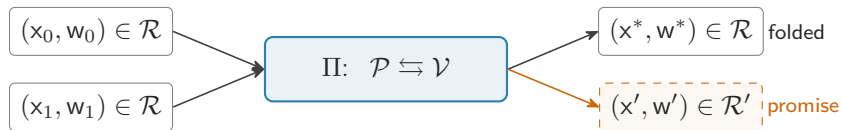
Sequentially, $\Pi_2 \circ \Pi_1 : \mathcal{R}_1 \rightarrow \mathcal{R}_2 \rightarrow \mathcal{R}_3$, and in parallel, $\Pi_1 \times \Pi_2 : \mathcal{R}_1 \times \mathcal{R}_3 \rightarrow \mathcal{R}_2 \times \mathcal{R}_4$.
Complex protocols are built from small, simple bricks.

Folding: two statements into one



A **promise folding scheme** is a reduction $\Pi : \mathcal{R} \times \mathcal{R} \rightarrow \mathcal{R} \times \mathcal{R}'$:

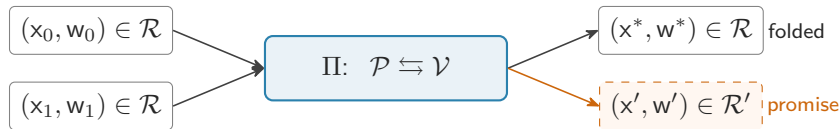
Folding: two statements into one



A **promise folding scheme** is a reduction $\Pi: \mathcal{R} \times \mathcal{R} \rightarrow \mathcal{R} \times \mathcal{R}'$:

- it **folds** two statements of $\mathcal{R}$ into **one** statement x^* of the same relation,

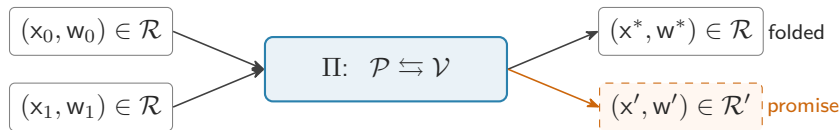
Folding: two statements into one



A **promise folding scheme** is a reduction $\Pi: \mathcal{R} \times \mathcal{R} \rightarrow \mathcal{R} \times \mathcal{R}'$:

- it **folds** two statements of $\mathcal{R}$ into **one** statement x^* of the same relation,
- and **postpones** some checks as a **promise** x' of another relation $\mathcal{R}'$, to be proven later, all promises together.

Folding: two statements into one

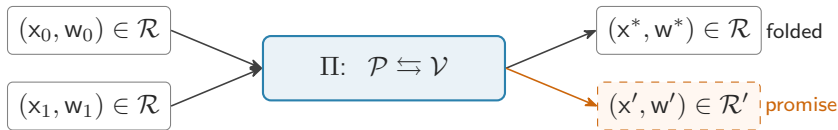


A **promise folding scheme** is a reduction $\Pi: \mathcal{R} \times \mathcal{R} \rightarrow \mathcal{R} \times \mathcal{R}'$:

- it **folds** two statements of $\mathcal{R}$ into **one** statement x^* of the same relation,
- and **postpones** some checks as a **promise** x' of another relation $\mathcal{R}'$, to be proven later, all promises together.

Proving x^* **and** the promise x' is enough to prove both x_0 and x_1 .

Folding: two statements into one



A **promise folding scheme** is a reduction $\Pi : \mathcal{R} \times \mathcal{R} \rightarrow \mathcal{R} \times \mathcal{R}'$:

- it **folds** two statements of $\mathcal{R}$ into **one** statement x^* of the same relation,
- and **postpones** some checks as a **promise** x' of another relation $\mathcal{R}'$, to be proven later, all promises together.

Proving x^* **and** the promise x' is enough to prove both x_0 and x_1 .

Plain folding: the trivial case

If $\mathcal{R}'$ is the **trivial relation** (always satisfied), there is nothing left to promise and the scheme is a plain folding scheme $\Pi : \mathcal{R} \times \mathcal{R} \rightarrow \mathcal{R}$.

Local folding: K statements into one

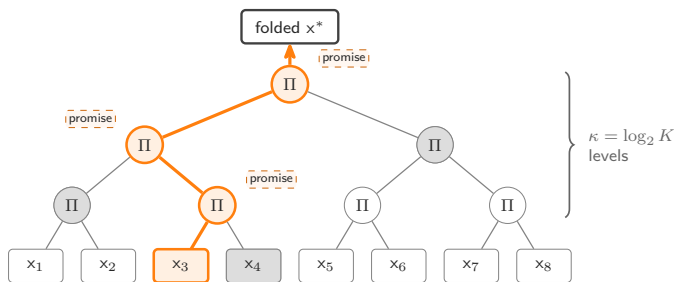
K -local folding scheme (simplified)

$\mathcal{P}$ maps K pairs (x_i, w_i) to one folded pair (x^*, w^*) plus, for each i , a proof π_i and promises;
 $\mathcal{V}(i, x_i, x^*, \pi_i, \text{promises}_i) \in \{0, 1\}$ reads only $o(K \cdot |x|)$ data.

Local folding: K statements into one

K -local folding scheme (simplified)

$\mathcal{P}$ maps K pairs (x_i, w_i) to one folded pair (x^*, w^*) plus, for each i , a proof π_i and promises;
 $\mathcal{V}(i, x_i, x^*, \pi_i, \text{promises}_i) \in \{0, 1\}$ reads only $o(K \cdot |x|)$ data.

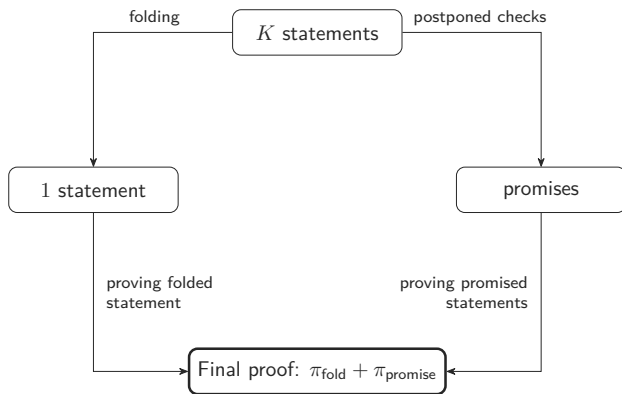


- Fold pairwise along a binary tree: $K = 2^\kappa$ statements become **one** after $\kappa = \log_2 K$ levels.
- Verifier i checks **only its path**: κ folds, fed by the shaded siblings, and κ promises.
- Its proof has size $\mathcal{O}(\log K)$, not $\mathcal{O}(K)$.

Outline

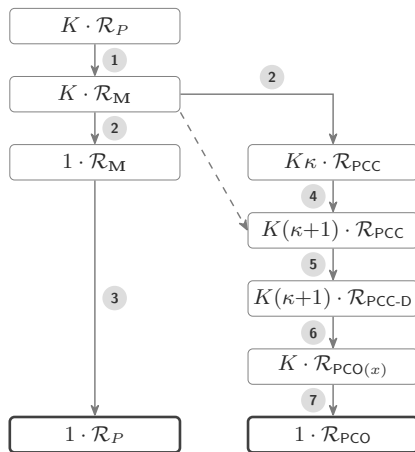
1. Introduction
2. Thesis motivation
3. Relations, reductions and folding
4. The protocol
5. Experimental results
6. Conclusions

Overview



- **Fold** the K input statements into **one** statement of the same type.
- Every fold **postpones** some checks: the **promises**.
- Prove the folded statement **once**, and all the promises **together**.
- Verifier i checks its fold path and its share of the promise proof.

The protocol in detail



Where the promises end: $\mathcal{R}_{\text{PCO}}$

Polynomial commitment opening

$$\mathcal{R}_{\text{PCO}} = \{ (x, w) = ((c, x, y), p) : c = \text{Com}(p), \deg p < D, p(x) = y \}$$

“ c is a commitment to a polynomial p that takes the value y at the point x .” $\mathcal{R}_{\text{PCO}(x)}$ is the same relation for a fixed point x .

Where the promises end: $\mathcal{R}_{\text{PCO}}$

Polynomial commitment opening

$$\mathcal{R}_{\text{PCO}} = \{ (x, w) = ((c, x, y), p) : c = \text{Com}(p), \deg p < D, p(x) = y \}$$

“ c is a commitment to a polynomial p that takes the value y at the point x .” $\mathcal{R}_{\text{PCO}(x)}$ is the same relation for a fixed point x .

- Steps 4–7 turn all the $K(\kappa + 1)$ promises into **a single** statement of $\mathcal{R}_{\text{PCO}}$, by checking them at a common random point x and merging the commitments (Com is homomorphic).

Where the promises end: $\mathcal{R}_{\text{PCO}}$

Polynomial commitment opening

$$\mathcal{R}_{\text{PCO}} = \{ (x, w) = ((c, x, y), p) : c = \text{Com}(p), \deg p < D, p(x) = y \}$$

“ c is a commitment to a polynomial p that takes the value y at the point x .” $\mathcal{R}_{\text{PCO}(x)}$ is the same relation for a fixed point x .

- Steps 4–7 turn all the $K(\kappa + 1)$ promises into **a single** statement of $\mathcal{R}_{\text{PCO}}$, by checking them at a common random point x and merging the commitments (Com is homomorphic).
- It is proven directly with **one KZG opening**: one group element, checked with one pairing.

Putting it together

What the Server outputs

- $1 \cdot \mathcal{R}_P$: proven once with Marlin's WI π .
- $1 \cdot \mathcal{R}_{PCO}$: one KZG opening.

Both are **independent of K** .

Putting it together

What the Server outputs

- $1 \cdot \mathcal{R}_P$: proven once with Marlin's WI π .
- $1 \cdot \mathcal{R}_{PCO}$: one KZG opening.

Both are **independent of K** .

What the Server pays

K statements for the price of **one** proof plus an $\mathcal{O}(K \log K)$ overhead of group operations.

Putting it together

What the Server outputs

- $1 \cdot \mathcal{R}_P$: proven once with Marlin's WI π .
- $1 \cdot \mathcal{R}_{PCO}$: one KZG opening.

Both are **independent of K** .

What the Server pays

K statements for the price of **one** proof plus an $\mathcal{O}(K \log K)$ overhead of group operations.

What verifier i checks

- its leaf encoding: $\mathcal{O}(1)$,
- its fold path: κ steps,
- its Merkle path: κ hashes,
- its share of the promise discharge,
- two pairings (Π_P and the final opening).

Total work and proof size: $\mathcal{O}(\log K)$.

Outline

1. Introduction
2. Thesis motivation
3. Relations, reductions and folding
4. The protocol
5. Experimental results
6. Conclusions

Expected costs

	K independent $WI \pi$	Amortization + 1 $WI \pi$
Server	$K (s \log s \mathbb{F} + s \mathbb{G})$	$(Ks + s \log s) \mathbb{F} + (K \log K + s) \mathbb{G}$
Each verifier	$\mathbb{F} + \mathbb{G} + \rho + P$	$2P + \log n \mathbb{F} + \log K (\mathbb{F} + \mathbb{G} + \rho)$
Proof size	λ	$\lambda \log K$

$\mathbb{G}$: group operation (expensive) $\mathbb{F}$: field operation (cheap) P : pairing ρ : hash λ : one group element
 $s = s_M$: non-zeros of the matrices

Expected costs

	K independent $WI \pi$	Amortization + 1 $WI \pi$
Server	$K (s \log s \mathbb{F} + s \mathbb{G})$	$(Ks + s \log s) \mathbb{F} + (K \log K + s) \mathbb{G}$
Each verifier	$\mathbb{F} + \mathbb{G} + \rho + P$	$2P + \log n \mathbb{F} + \log K (\mathbb{F} + \mathbb{G} + \rho)$
Proof size	λ	$\lambda \log K$

$\mathbb{G}$: group operation (expensive) $\mathbb{F}$: field operation (cheap) P : pairing ρ : hash λ : one group element
 $s = s_M$: non-zeros of the matrices

- Server:** the expensive group operations go from $K \cdot s$ to $K \log K + s$. When $s \gg K$, the Server pays about **one** proof.

Expected costs

	K independent $WI \pi$	Amortization + 1 $WI \pi$
Server	$K (s \log s \mathbb{F} + s \mathbb{G})$	$(Ks + s \log s) \mathbb{F} + (K \log K + s) \mathbb{G}$
Each verifier	$\mathbb{F} + \mathbb{G} + \rho + P$	$2P + \log n \mathbb{F} + \log K (\mathbb{F} + \mathbb{G} + \rho)$
Proof size	λ	$\lambda \log K$

$\mathbb{G}$: group operation (expensive) $\mathbb{F}$: field operation (cheap) P : pairing ρ : hash λ : one group element
 $s = s_M$: non-zeros of the matrices

- **Server:** the expensive group operations go from $K \cdot s$ to $K \log K + s$. When $s \gg K$, the Server pays about **one** proof.
- **Verifier:** from constant to **logarithmic in K** , independent of the circuit size.

Experimental setup

What we compare (Server side):

- Side 1: K independent Marlin WI π .
- Side 2: amortization of the K statements
+ 1 WI π .

Rounds 1–2 (witness-dependent) are identical on both sides, so they are not measured.

Experimental setup

What we compare (Server side):

- **Side 1:** K independent Marlin WI π .
- **Side 2:** amortization of the K statements
+ 1 WI π .

Rounds 1–2 (witness-dependent) are identical on both sides, so they are not measured.

Parameters

- circuit size $n \in \{2^{10}, 2^{12}, \dots, 2^{20}\}$,
- $K \in \{1, 2, 4, \dots, 128\}$ verifiers,
- random R1CS with $s_M = 2n$ non-zeros.

Experimental setup

What we compare (Server side):

- **Side 1:** K independent Marlin $\text{WI } \pi$.
- **Side 2:** amortization of the K statements + 1 $\text{WI } \pi$.

Rounds 1–2 (witness-dependent) are identical on both sides, so they are not measured.

Parameters

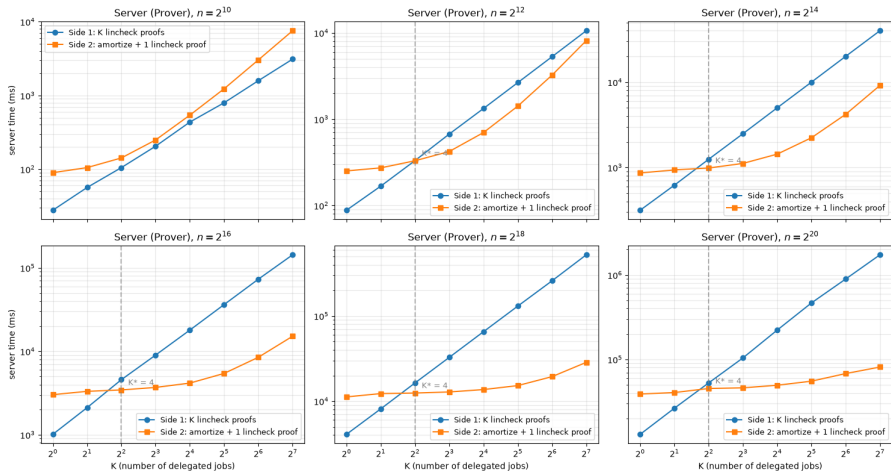
- circuit size $n \in \{2^{10}, 2^{12}, \dots, 2^{20}\}$,
- $K \in \{1, 2, 4, \dots, 128\}$ verifiers,
- random R1CS with $s_{\mathbf{M}} = 2n$ non-zeros.

Machine

One laptop: AMD Ryzen 9 270 (8 cores, 16 threads), 64 GB DDR5, Windows 11, Rust 1.94 (release).

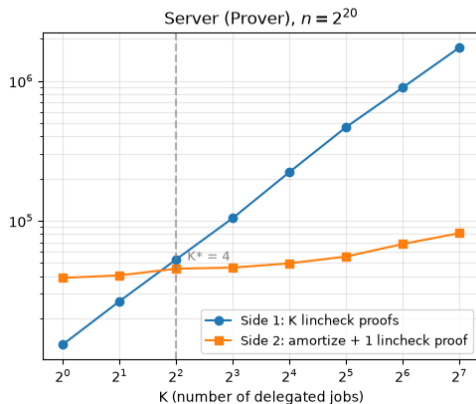
Both sides run on the same machine: the **ratios** matter, not the absolute times.

Server (prover) time



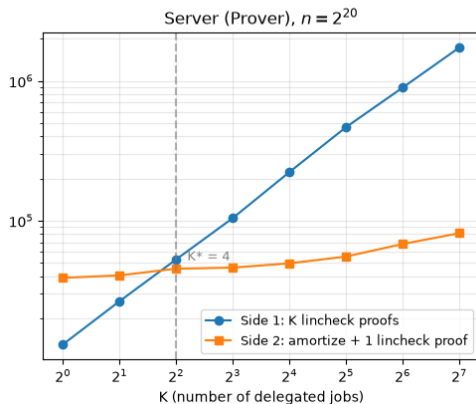
blue: K independent $WL \pi$ orange: amortization + 1 $WL \pi$ (time in ms, log scale)

Server time: the numbers



- The curves **cross at** $K = 4$ for every $n \geq 2^{12}$.
- Past that, the gain grows with the circuit. At $K = 128$:

Server time: the numbers



- The curves **cross at** $K = 4$ for every $n \geq 2^{12}$.
- Past that, the gain grows with the circuit. At $K = 128$:

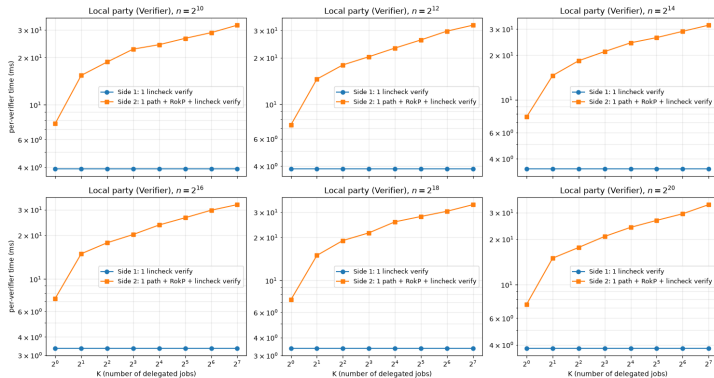
n	2^{12}	2^{16}	2^{20}
speedup	$\times 1.3$	$\times 9.5$	$\times 21$

$K = 128$, $n = 2^{20}$ (left)

$1.7 \cdot 10^6$ ms (≈ 28 min) $\rightarrow 8.1 \cdot 10^4$ ms (≈ 81 s):
about **95% less** work for the Server.

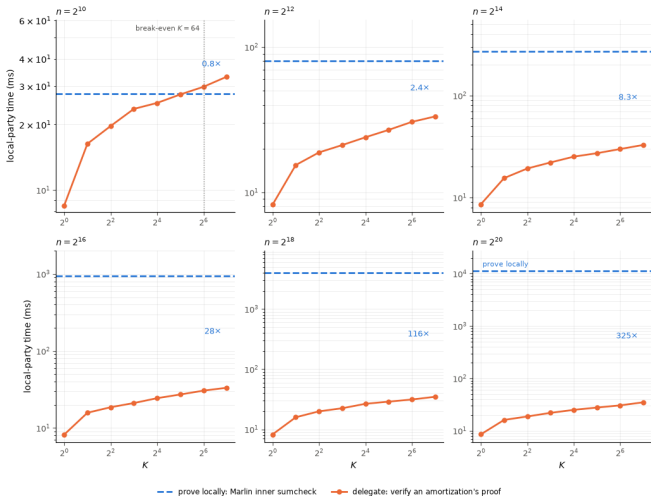
At $n = 2^{10}$ the circuit is too small: the amortized side stays above the baseline for every K .

Verifier time



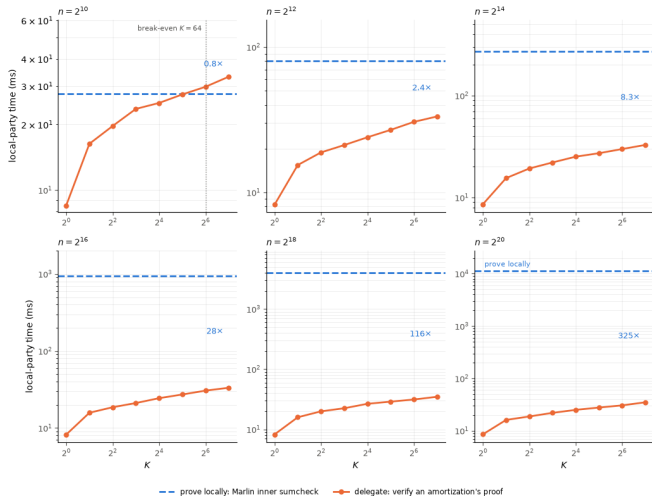
- **Baseline:** ≈ 4 ms, constant in K and in n (the $W\Gamma\pi$ verifier is succinct).
- **Amortized:** ≈ 8 ms at $K = 1$ (an extra pairing, from Π_P) up to ≈ 35 ms at $K = 128$, **about $\times 9$:** grows as $\log K$, independent of n .

Is delegating worth it for the user?



- **Dashed:** the user proves its own $\mathcal{R}_P$ claim (Marlin's WI π). Grows with n .
- **Solid:** the user delegates and verifies the amortized proof. Grows only with $\log K$.

Is delegating worth it for the user?



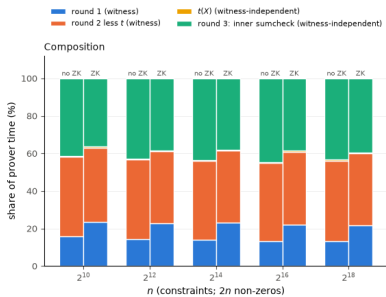
- **Dashed:** the user proves its own $\mathcal{R}_P$ claim (Marlin's WI π). Grows with n .
- **Solid:** the user delegates and verifies the amortized proof. Grows only with $\log K$.

At $K = 128$, delegating is cheaper by

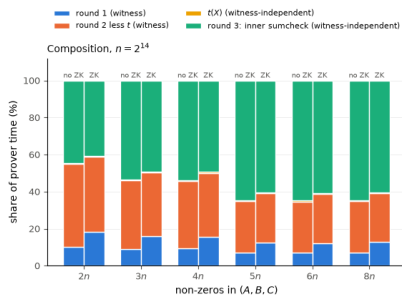
n	2^{12}	2^{16}	2^{20}
gain	$\times 2.4$	$\times 28$	$\times \mathbf{325}$

At $n = 2^{10}$, proving locally is better from $K = 64$ on.

How much of Marlin can be delegated?

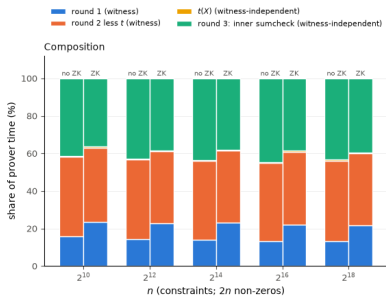


varying the circuit size n ($2n$ non-zeros)

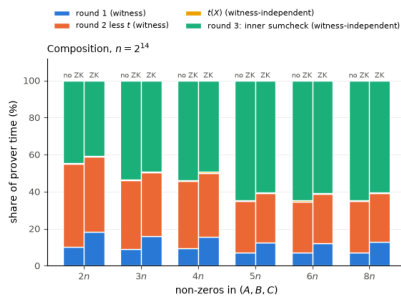


varying the density, $n = 2^{14}$

How much of Marlin can be delegated?



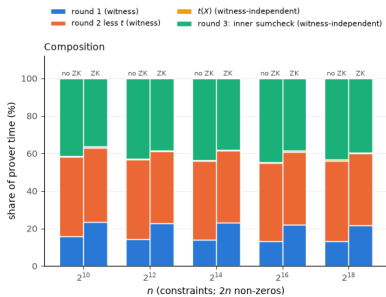
varying the circuit size n ($2n$ non-zeros)



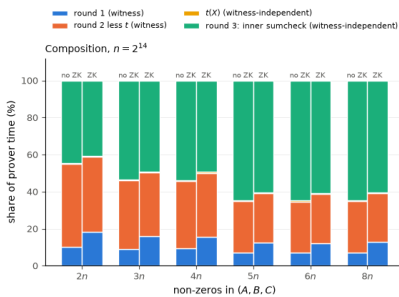
varying the density, $n = 2^{14}$

- The witness-independent part (green) is $\approx 40\%$ of Marlin's proving time, **whatever the circuit size**.

How much of Marlin can be delegated?



varying the circuit size n ($2n$ non-zeros)



varying the density, $n = 2^{14}$

- The witness-independent part (green) is $\approx 40\%$ of Marlin's proving time, **whatever the circuit size**.
- It grows with the density, up to $\approx 60\%$ with $8n$ non-zeros; zero-knowledge barely changes it.

Outline

1. Introduction
2. Thesis motivation
3. Relations, reductions and folding
4. The protocol
5. Experimental results
6. Conclusions

Conclusions

The amortization protocol of Paslis, Ràfols and Zacharakis delivers the expected gains, now measured on a **full implementation** over Marlin.

Conclusions

The amortization protocol of Paslis, Ràfols and Zacharakis delivers the expected gains, now measured on a **full implementation** over Marlin.

- **Server:** group operations drop from $K \cdot s_M$ to $K \log K + s_M$ (when $s_M \gg K$).
 - $K = 128, n = 2^{20}$: $1.7 \cdot 10^6$ ms $\rightarrow$ $8.1 \cdot 10^4$ ms, $\approx 95\%$ **less**.
 - Pays off from $K = 4$ for every $n \geq 2^{12}$; the gain grows with the circuit, from $\times 1.3$ to $\times 21$.

Conclusions

The amortization protocol of Paslis, Ràfols and Zacharakis delivers the expected gains, now measured on a **full implementation** over Marlin.

- **Server:** group operations drop from $K \cdot s_M$ to $K \log K + s_M$ (when $s_M \gg K$).
 - $K = 128, n = 2^{20}$: $1.7 \cdot 10^6$ ms $\rightarrow$ $8.1 \cdot 10^4$ ms, $\approx 95\%$ **less**.
 - Pays off from $K = 4$ for every $n \geq 2^{12}$; the gain grows with the circuit, from $\times 1.3$ to $\times 21$.
- **Verifier:** from constant to logarithmic in K : ≈ 4 ms $\rightarrow \approx 35$ ms (about $\times 9$), independent of the circuit size, and still up to $\times 325$ **cheaper** than proving locally.

Conclusions

The amortization protocol of Paslis, Ràfols and Zacharakis delivers the expected gains, now measured on a **full implementation** over Marlin.

- **Server:** group operations drop from $K \cdot s_M$ to $K \log K + s_M$ (when $s_M \gg K$).
 - $K = 128, n = 2^{20}$: $1.7 \cdot 10^6$ ms $\rightarrow$ $8.1 \cdot 10^4$ ms, $\approx 95\%$ **less**.
 - Pays off from $K = 4$ for every $n \geq 2^{12}$; the gain grows with the circuit, from $\times 1.3$ to $\times 21$.
- **Verifier:** from constant to logarithmic in K : ≈ 4 ms $\rightarrow \approx 35$ ms (about $\times 9$), independent of the circuit size, and still up to $\times 325$ **cheaper** than proving locally.
- **Delegation is worth it:** 40–60% of Marlin's proving time is witness-independent.

Future work

- **Other SNARKs** whose witness-independent part is also a bivariate polynomial evaluation, such as Lunar or Basilisk: measure the gains there.

Future work

- **Other SNARKs** whose witness-independent part is also a bivariate polynomial evaluation, such as Lunar or Basilisk: measure the gains there.
- **Other commitment families:**
 - multivariate commitments (Spartan, Samaritan),
 - hash-based, non-homomorphic commitments (Fractal).

Future work

- **Other SNARKs** whose witness-independent part is also a bivariate polynomial evaluation, such as Lunar or Basilisk: measure the gains there.
- **Other commitment families:**
 - multivariate commitments (Spartan, Samaritan),
 - hash-based, non-homomorphic commitments (Fractal).
- **End-to-end delegation:** collaborative SNARKs for the witness-dependent part, combined with the amortized witness-independent part.

Thank you!

Questions?

Antonio Peso Vilella

`github.com/apesvil/amortized-proofs-masters-thesis`



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH
Facultat de Matemàtiques i Estadística



The relation we want to amortize

Encode the contract's matrix $\mathbf{M} \in \mathbb{F}^{n \times n}$ as a bivariate polynomial over a domain $\mathbb{H} = \{h_1, \dots, h_n\} \subset \mathbb{F}$: $P(h_i, h_j) = \mathbf{M}_{ij}$.

The relation we want to amortize

Encode the contract's matrix $\mathbf{M} \in \mathbb{F}^{n \times n}$ as a bivariate polynomial over a domain $\mathbb{H} = \{h_1, \dots, h_n\} \subset \mathbb{F}$: $P(h_i, h_j) = \mathbf{M}_{ij}$.

Input: bivariate evaluation

$$\mathcal{R}_P = \{ (\alpha, \beta, y) : P(\alpha, \beta) = y \}$$

No witness: the claim is public, but checking it directly costs $\approx s_{\mathbf{M}}$ operations.

The relation we want to amortize

Encode the contract's matrix $\mathbf{M} \in \mathbb{F}^{n \times n}$ as a bivariate polynomial over a domain $\mathbb{H} = \{h_1, \dots, h_n\} \subset \mathbb{F}$: $P(h_i, h_j) = \mathbf{M}_{ij}$.

Input: bivariate evaluation

$$\mathcal{R}_P = \{(\alpha, \beta, y) : P(\alpha, \beta) = y\}$$

No witness: the claim is public, but checking it directly costs $\approx s_{\mathbf{M}}$ operations.

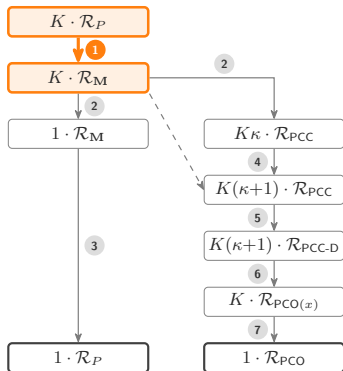
Matrix form

With the Lagrange basis

$\boldsymbol{\lambda}(X) = (\lambda_1(X), \dots, \lambda_n(X))$ of $\mathbb{H}$:

$$P(\alpha, \beta) = \boldsymbol{\lambda}(\alpha)^\top \mathbf{M} \boldsymbol{\lambda}(\beta).$$

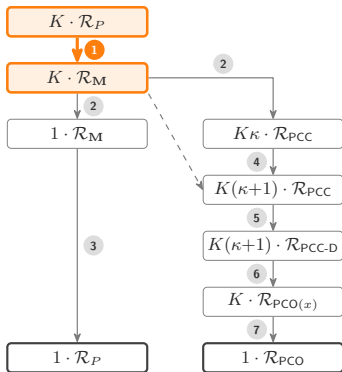
Step 1: encode each leaf



$$K \cdot \mathcal{R}_P \rightarrow K \cdot \mathcal{R}_M$$

Verifier i holds (α_i, β_i, y_i) with $P(\alpha_i, \beta_i) = y_i$.

Step 1: encode each leaf

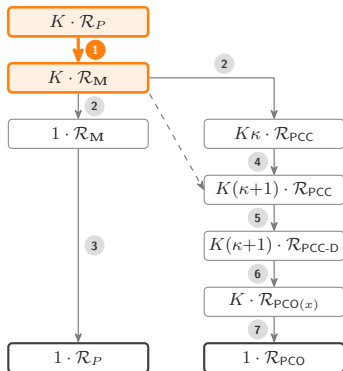


$$K \cdot \mathcal{R}_P \rightarrow K \cdot \mathcal{R}_M$$

Verifier i holds (α_i, β_i, y_i) with $P(\alpha_i, \beta_i) = y_i$.

- Matrix form: $\mathbf{u}_i = \lambda(\alpha_i)$ and $\mathbf{v}_i = \lambda(\beta_i)$, so $y_i = \mathbf{u}_i^\top \mathbf{M} \mathbf{v}_i$.

Step 1: encode each leaf



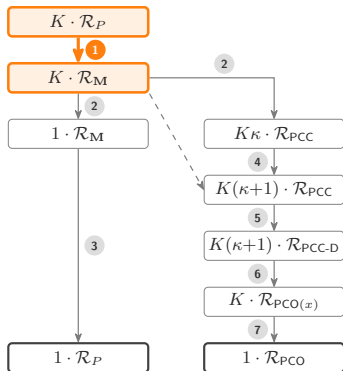
$$K \cdot \mathcal{R}_P \rightarrow K \cdot \mathcal{R}_M$$

Verifier i holds (α_i, β_i, y_i) with $P(\alpha_i, \beta_i) = y_i$.

- Matrix form: $\mathbf{u}_i = \boldsymbol{\lambda}(\alpha_i)$ and $\mathbf{v}_i = \boldsymbol{\lambda}(\beta_i)$, so $y_i = \mathbf{u}_i^\top \mathbf{M} \mathbf{v}_i$.
- Closed form of the committed polynomial, $\mathbf{u}_i^\top \boldsymbol{\lambda}(X) = N(X)/n T(X)$, with

$$N(X) = (X^n - 1) \alpha_i - (\alpha_i^n - 1) X, \quad T(X) = X - \alpha_i.$$

Step 1: encode each leaf



$$K \cdot \mathcal{R}_P \rightarrow K \cdot \mathcal{R}_M$$

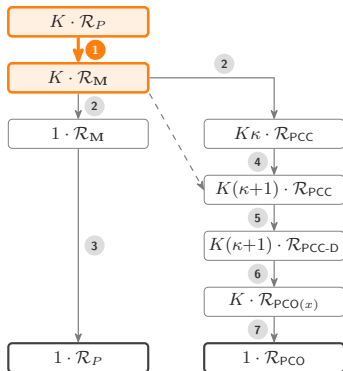
Verifier i holds (α_i, β_i, y_i) with $P(\alpha_i, \beta_i) = y_i$.

- Matrix form: $\mathbf{u}_i = \boldsymbol{\lambda}(\alpha_i)$ and $\mathbf{v}_i = \boldsymbol{\lambda}(\beta_i)$, so $y_i = \mathbf{u}_i^\top \mathbf{M} \mathbf{v}_i$.
- Closed form of the committed polynomial, $\mathbf{u}_i^\top \boldsymbol{\lambda}(X) = N(X)/n T(X)$, with

$$N(X) = (X^n - 1) \alpha_i - (\alpha_i^n - 1) X, \quad T(X) = X - \alpha_i.$$

- Only $3 + 2$ coefficients: committing costs $\mathcal{O}(1)$ group operations, and $\mathcal{V}_i$ **recomputes the commitments itself**.

Step 1: encode each leaf



$$K \cdot \mathcal{R}_P \rightarrow K \cdot \mathcal{R}_M$$

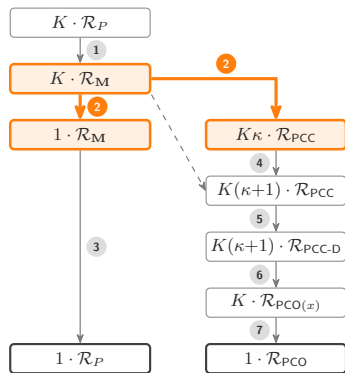
Verifier i holds (α_i, β_i, y_i) with $P(\alpha_i, \beta_i) = y_i$.

- Matrix form: $\mathbf{u}_i = \boldsymbol{\lambda}(\alpha_i)$ and $\mathbf{v}_i = \boldsymbol{\lambda}(\beta_i)$, so $y_i = \mathbf{u}_i^\top \mathbf{M} \mathbf{v}_i$.
- Closed form of the committed polynomial, $\mathbf{u}_i^\top \boldsymbol{\lambda}(X) = N(X)/n T(X)$, with

$$N(X) = (X^n - 1) \alpha_i - (\alpha_i^n - 1) X, \quad T(X) = X - \alpha_i.$$

- Only $3 + 2$ coefficients: committing costs $\mathcal{O}(1)$ group operations, and $\mathcal{V}_i$ **recomputes the commitments itself**.
- That the encoding is consistent is **promised**: one $\mathcal{R}_{\text{PCC}}$ per leaf, set aside until step 4.

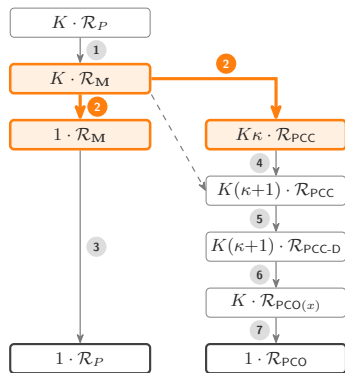
Step 2: fold



$$K \cdot \mathcal{R}_M \rightarrow 1 \cdot \mathcal{R}_M + K\kappa \cdot \mathcal{R}_{PCC}$$

$$\mathcal{L}_{PCC}(\text{FS}(\Pi_M), \kappa)$$

Step 2: fold

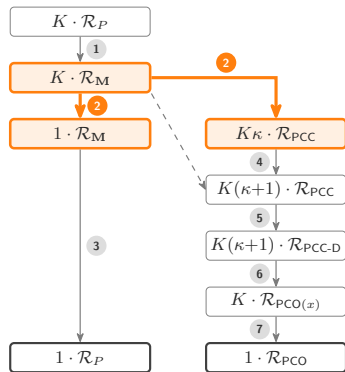


$$K \cdot \mathcal{R}_M \rightarrow 1 \cdot \mathcal{R}_M + K\kappa \cdot \mathcal{R}_{PCC}$$

$$L_{PCC}(\text{FS}(\Pi_M), \kappa)$$

- Run Π_M at every node of a binary tree over the K leaves.

Step 2: fold

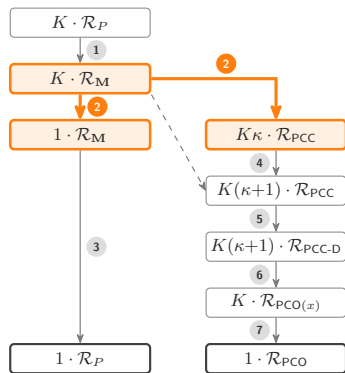


$$K \cdot \mathcal{R}_M \rightarrow 1 \cdot \mathcal{R}_M + K\kappa \cdot \mathcal{R}_{PCC}$$

$$\mathcal{L}_{PCC}(\text{FS}(\Pi_M), \kappa)$$

- Run Π_M at every node of a binary tree over the K leaves.
- Each node: cross term z , challenge γ , folded $\mathbf{u}^*, \mathbf{v}^*, y^*$, and **one promise**.

Step 2: fold

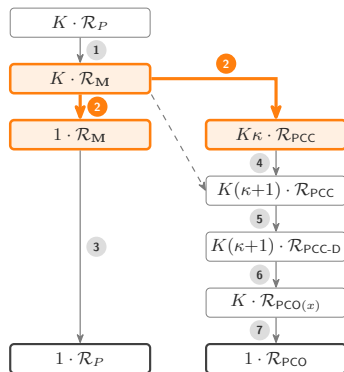


$$K \cdot \mathcal{R}_M \rightarrow 1 \cdot \mathcal{R}_M + K\kappa \cdot \mathcal{R}_{PCC}$$

$$\mathcal{L}_{PCC}(\text{FS}(\Pi_M), \kappa)$$

- Run Π_M at every node of a binary tree over the K leaves.
- Each node: cross term z , challenge γ , folded $\mathbf{u}^*, \mathbf{v}^*, y^*$, and **one promise**.
- No interaction: γ is a **hash** of the node's inputs (Fiat–Shamir).

Step 2: fold

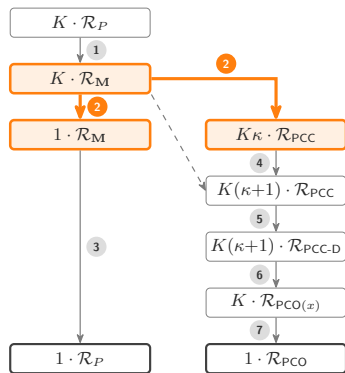


$$K \cdot \mathcal{R}_M \rightarrow 1 \cdot \mathcal{R}_M + K\kappa \cdot \mathcal{R}_{PCC}$$

$$\mathcal{L}_{PCC}(\text{FS}(\Pi_M), \kappa)$$

- Run Π_M at every node of a binary tree over the K leaves.
- Each node: cross term z , challenge γ , folded $\mathbf{u}^*, \mathbf{v}^*, y^*$, and **one promise**.
- No interaction: γ is a **hash** of the node's inputs (Fiat–Shamir).
- $\mathcal{V}_i$ receives the κ node proofs on its path and checks $\kappa = \log_2 K$ folds.

Step 2: fold

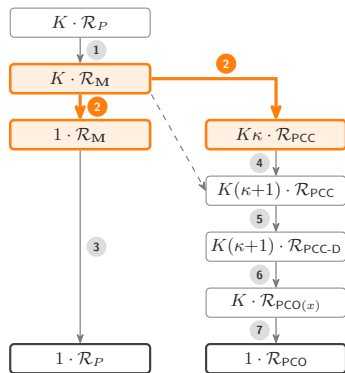


$$K \cdot \mathcal{R}_M \rightarrow 1 \cdot \mathcal{R}_M + K\kappa \cdot \mathcal{R}_{PCC}$$

$$\mathcal{L}_{PCC}(\text{FS}(\Pi_M), \kappa)$$

- Run Π_M at every node of a binary tree over the K leaves.
- Each node: cross term z , challenge γ , folded $\mathbf{u}^*, \mathbf{v}^*, y^*$, and **one promise**.
- No interaction: γ is a **hash** of the node's inputs (Fiat–Shamir).
- $\mathcal{V}_i$ receives the κ node proofs on its path and checks $\kappa = \log_2 K$ folds.
- Server: commits to the sparse N^*, T^* of the $K - 1$ nodes, in $\mathcal{O}(K \log K)$ group operations.

Step 2: fold



$$K \cdot \mathcal{R}_M \rightarrow 1 \cdot \mathcal{R}_M + K\kappa \cdot \mathcal{R}_{PCC}$$

$$\mathcal{L}_{PCC}(\text{FS}(\Pi_M), \kappa)$$

- Run Π_M at every node of a binary tree over the K leaves.
- Each node: cross term z , challenge γ , folded $\mathbf{u}^*, \mathbf{v}^*, y^*$, and **one promise**.
- No interaction: γ is a **hash** of the node's inputs (Fiat–Shamir).
- $\mathcal{V}_i$ receives the κ node proofs on its path and checks $\kappa = \log_2 K$ folds.
- Server: commits to the sparse N^*, T^* of the $K - 1$ nodes, in $\mathcal{O}(K \log K)$ group operations.

Step 2: inside one fold

$$\mathcal{R}_M \times \mathcal{R}_M \rightarrow \mathcal{R}_M \times \mathcal{R}_{\text{PCC}}$$

 Π_M

Step 2: inside one fold

$$\mathcal{R}_M \times \mathcal{R}_M \rightarrow \mathcal{R}_M \times \mathcal{R}_{PCC}$$

 Π_M

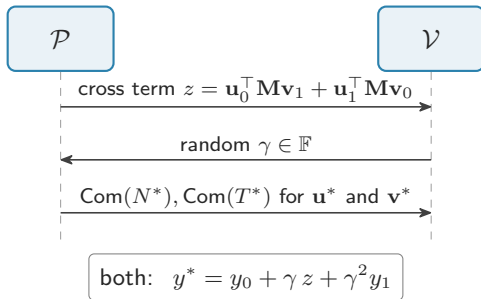
Input: $y_b = \mathbf{u}_b^\top \mathbf{M} \mathbf{v}_b$ for $b \in \{0, 1\}$, where $\mathcal{V}$ only holds commitments to the sparse encodings $\mathbf{u}_b^\top \boldsymbol{\lambda}(X) = N_b(X)/n T_b(X)$ (likewise for $\mathbf{v}_b$).

Step 2: inside one fold

$$\mathcal{R}_M \times \mathcal{R}_M \rightarrow \mathcal{R}_M \times \mathcal{R}_{\text{PCC}}$$

 Π_M

Input: $y_b = \mathbf{u}_b^\top \mathbf{M} \mathbf{v}_b$ for $b \in \{0, 1\}$, where $\mathcal{V}$ only holds commitments to the sparse encodings $\mathbf{u}_b^\top \boldsymbol{\lambda}(X) = N_b(X)/n T_b(X)$ (likewise for $\mathbf{v}_b$).

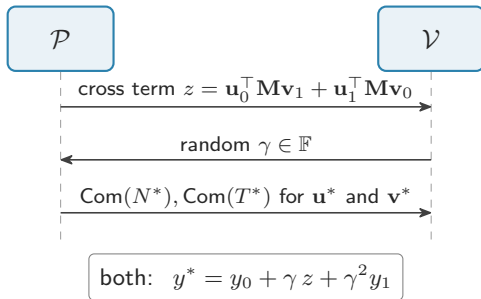


Step 2: inside one fold

$$\mathcal{R}_M \times \mathcal{R}_M \rightarrow \mathcal{R}_M \times \mathcal{R}_{\text{PCC}}$$

 Π_M

Input: $y_b = \mathbf{u}_b^\top \mathbf{M} \mathbf{v}_b$ for $b \in \{0, 1\}$, where $\mathcal{V}$ only holds commitments to the sparse encodings $\mathbf{u}_b^\top \lambda(X) = N_b(X)/n T_b(X)$ (likewise for $\mathbf{v}_b$).



Folded statement: $\mathcal{R}_M$

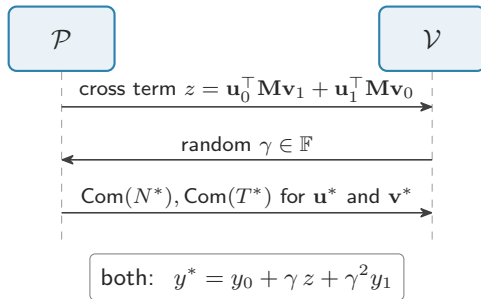
$\mathbf{u}^* = \mathbf{u}_0 + \gamma \mathbf{u}_1$ and $\mathbf{v}^* = \mathbf{v}_0 + \gamma \mathbf{v}_1$ satisfy
 $\mathbf{u}^{*\top} \mathbf{M} \mathbf{v}^* = y_0 + \gamma z + \gamma^2 y_1 = y^*.$

Step 2: inside one fold

$$\mathcal{R}_M \times \mathcal{R}_M \rightarrow \mathcal{R}_M \times \mathcal{R}_{PCC}$$

Π_M

Input: $y_b = \mathbf{u}_b^\top \mathbf{M} \mathbf{v}_b$ for $b \in \{0, 1\}$, where $\mathcal{V}$ only holds commitments to the sparse encodings $\mathbf{u}_b^\top \lambda(X) = N_b(X)/n T_b(X)$ (likewise for $\mathbf{v}_b$).



Folded statement: $\mathcal{R}_M$

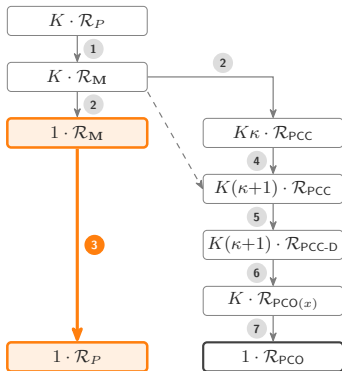
$\mathbf{u}^* = \mathbf{u}_0 + \gamma \mathbf{u}_1$ and $\mathbf{v}^* = \mathbf{v}_0 + \gamma \mathbf{v}_1$ satisfy
 $\mathbf{u}^{*\top} \mathbf{M} \mathbf{v}^* = y_0 + \gamma z + \gamma^2 y_1 = y^*.$

Promise: $\mathcal{R}_{PCC}$

$N^* = N_0 T_1 + \gamma N_1 T_0$ and $T^* = T_0 T_1$ are **products**, so $\mathcal{V}$ cannot derive their commitments.
 $\mathcal{P}$ promises

$N^* - N_0 T_1 - \gamma N_1 T_0 = 0, \quad T^* - T_0 T_1 = 0$
 for $\mathbf{u}$ and $\mathbf{v}$ (plus degree bounds).

Step 3: back to the input relation

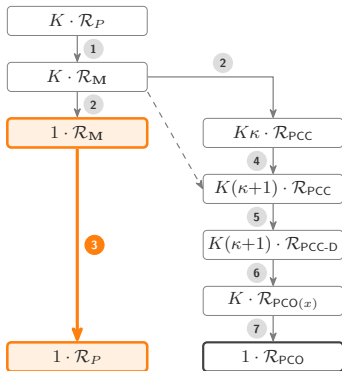


$$1 \cdot \mathcal{R}_M \rightarrow 1 \cdot \mathcal{R}_P$$

$\text{FS}(\Pi_P)$

The folded $\mathbf{u}^*, \mathbf{v}^*$ are no longer of the form $\lambda(\alpha), \lambda(\beta)$: turn the claim back into one the SNARK understands.

Step 3: back to the input relation



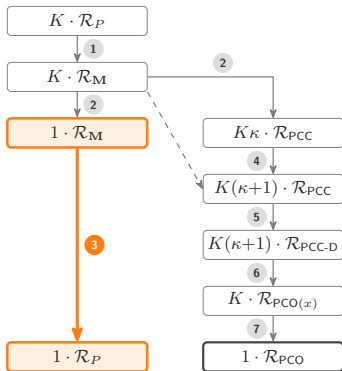
$$1 \cdot \mathcal{R}_M \rightarrow 1 \cdot \mathcal{R}_P$$

$\text{FS}(\Pi_P)$

The folded $\mathbf{u}^*, \mathbf{v}^*$ are no longer of the form $\lambda(\alpha), \lambda(\beta)$: turn the claim back into one the SNARK understands.

- With $\mathbf{w} = \mathbf{M}\mathbf{v}^*$, split the claim into $y^* = \mathbf{u}^{*\top} \mathbf{w}$ and $\mathbf{w} = \mathbf{M}\mathbf{v}^*$.

Step 3: back to the input relation



$$1 \cdot \mathcal{R}_M \rightarrow 1 \cdot \mathcal{R}_P$$

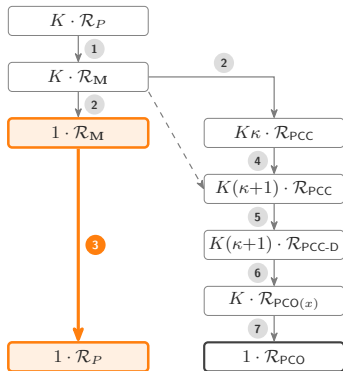
FS(Π_P)

The folded $\mathbf{u}^*, \mathbf{v}^*$ are no longer of the form $\lambda(\alpha), \lambda(\beta)$: turn the claim back into one the SNARK understands.

- With $\mathbf{w} = \mathbf{M}\mathbf{v}^*$, split the claim into $y^* = \mathbf{u}^{*\top} \mathbf{w}$ and $\mathbf{w} = \mathbf{M}\mathbf{v}^*$.
- Both are sums over $\mathbb{H}$, proven by a **univariate sumcheck**:

$$\sum_{h \in \mathbb{H}} f(h) = 0 \iff f(X) = X g(X) + (X^n - 1) q(X).$$

Step 3: back to the input relation



$$1 \cdot \mathcal{R}_M \rightarrow 1 \cdot \mathcal{R}_P$$

FS(Π_P)

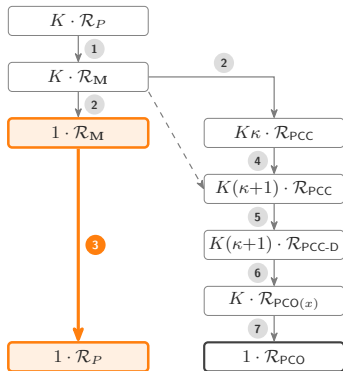
The folded $\mathbf{u}^*, \mathbf{v}^*$ are no longer of the form $\lambda(\alpha), \lambda(\beta)$: turn the claim back into one the SNARK understands.

- With $\mathbf{w} = \mathbf{M}\mathbf{v}^*$, split the claim into $y^* = \mathbf{u}^{*\top} \mathbf{w}$ and $\mathbf{w} = \mathbf{M}\mathbf{v}^*$.
- Both are sums over $\mathbb{H}$, proven by a **univariate sumcheck**:

$$\sum_{h \in \mathbb{H}} f(h) = 0 \iff f(X) = X g(X) + (X^n - 1) q(X).$$

- Output: a fresh claim $P(\alpha', \beta') = y'$ at new random points, of the **same type as the inputs**.

Step 3: back to the input relation



$$1 \cdot \mathcal{R}_M \rightarrow 1 \cdot \mathcal{R}_P$$

FS(Π_P)

The folded $\mathbf{u}^*, \mathbf{v}^*$ are no longer of the form $\lambda(\alpha), \lambda(\beta)$: turn the claim back into one the SNARK understands.

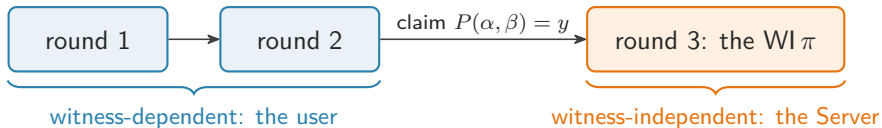
- With $\mathbf{w} = \mathbf{M}\mathbf{v}^*$, split the claim into $y^* = \mathbf{u}^{*\top} \mathbf{w}$ and $\mathbf{w} = \mathbf{M}\mathbf{v}^*$.
- Both are sums over $\mathbb{H}$, proven by a **univariate sumcheck**:

$$\sum_{h \in \mathbb{H}} f(h) = 0 \iff f(X) = X g(X) + (X^n - 1) q(X).$$

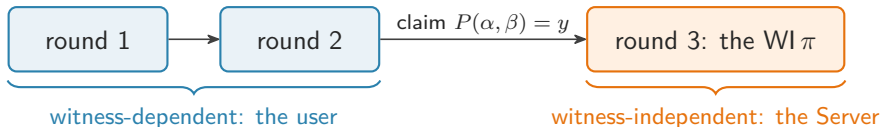
- Output: a fresh claim $P(\alpha', \beta') = y'$ at new random points, of the **same type as the inputs**.

This single $\mathcal{R}_P$ claim is proven **once** with Marlin's WI π , instead of K times.

Instantiation: the Marlin SNARK

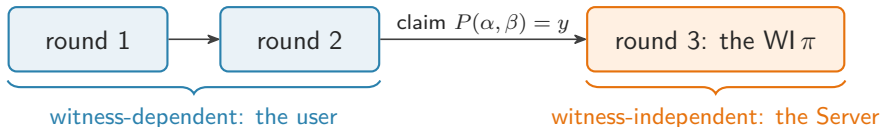


Instantiation: the Marlin SNARK



- **Marlin** (Chiesa et al., EUROCRYPT 2020): a SNARK for R1CS, $\mathbf{Az} \circ \mathbf{Bz} = \mathbf{Cz}$.

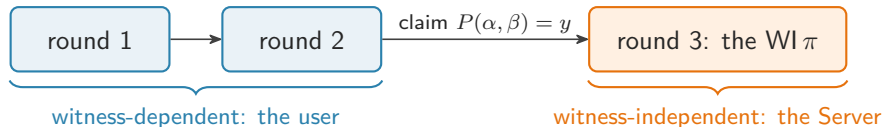
Instantiation: the Marlin SNARK



- **Marlin** (Chiesa et al., EUROCRYPT 2020): a SNARK for R1CS, $\mathbf{Az} \circ \mathbf{Bz} = \mathbf{Cz}$.
- Rounds 1–2 use the witness and end with a claim on the **public** matrices at random α, β, η , an $\mathcal{R}_P$ statement:

$$y = P_{\mathbf{A}}(\alpha, \beta) + \eta P_{\mathbf{B}}(\alpha, \beta) + \eta^2 P_{\mathbf{C}}(\alpha, \beta).$$

Instantiation: the Marlin SNARK

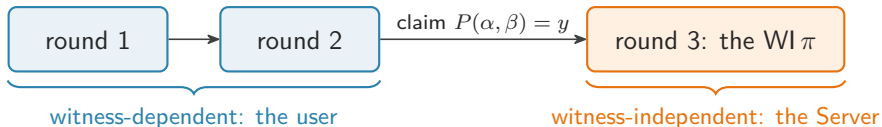


- **Marlin** (Chiesa et al., EUROCRYPT 2020): a SNARK for R1CS, $\mathbf{A}\mathbf{z} \circ \mathbf{B}\mathbf{z} = \mathbf{C}\mathbf{z}$.
- Rounds 1–2 use the witness and end with a claim on the **public** matrices at random α, β, η , an $\mathcal{R}_P$ statement:

$$y = P_{\mathbf{A}}(\alpha, \beta) + \eta P_{\mathbf{B}}(\alpha, \beta) + \eta^2 P_{\mathbf{C}}(\alpha, \beta).$$

- K users of one contract give K such statements on the same $\mathbf{A}, \mathbf{B}, \mathbf{C}$: the input of the amortization.

Instantiation: the Marlin SNARK



- **Marlin** (Chiesa et al., EUROCRYPT 2020): a SNARK for R1CS, $\mathbf{A}\mathbf{z} \circ \mathbf{B}\mathbf{z} = \mathbf{C}\mathbf{z}$.
- Rounds 1–2 use the witness and end with a claim on the **public** matrices at random α, β, η , an $\mathcal{R}_P$ statement:

$$y = P_{\mathbf{A}}(\alpha, \beta) + \eta P_{\mathbf{B}}(\alpha, \beta) + \eta^2 P_{\mathbf{C}}(\alpha, \beta).$$

- K users of one contract give K such statements on the same $\mathbf{A}, \mathbf{B}, \mathbf{C}$: the input of the amortization.

Implementation: Rust, arkworks, BLS12-381, KZG, Blake3. The three matrices are folded together (same commitments, three values).